

The Robertson Chemical Kinetics Problem

Stiff ODEs: how one fast process pins the global step size

Group 5 (names / student IDs to be filled in)

Numerical Analysis of ODEs and PDEs — Project 1, Topic 3

September 22, 2026

Three chemical species — how much of each is there?

A three-species reaction system (the Robertson problem)

$$\begin{aligned}y_1' &= -0.04 y_1 + 10^4 y_2 y_3 \\y_2' &= 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \\y_3' &= 3 \times 10^7 y_2^2 \\y(0) &= (1, 0, 0)^T, \quad t \in [0, 40]\end{aligned}$$

In one sentence

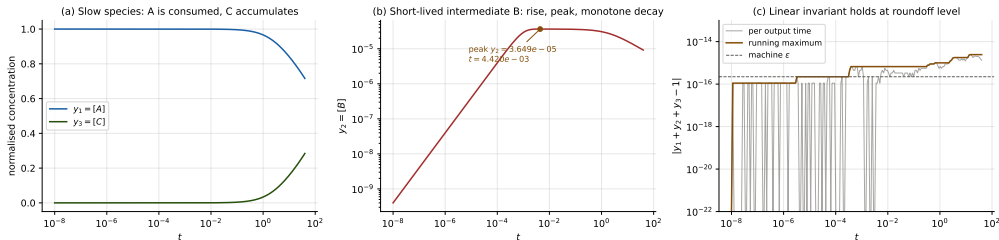
Species A, B and C interconvert, and at the start everything is A. We want to know how much of each is present at **every instant** of the 0–40 second window.

What we have to hand in

- three concentration curves $y_1(t), y_2(t), y_3(t)$
- one number: $y_2(40)$
- plus an argument: *why you should believe it*

What the answer looks like: one sharp spike, two slow curves

F2 Tight-tolerance reference (Radau, tightened tolerances)



Start with B (the middle curve)

B shoots up from zero, peaks at 3.65×10^{-5} at $t = 4.42 \times 10^{-3}$ s, then decays by 5 orders of magnitude, down to 9.18553×10^{-6} at $t = 40$ s.

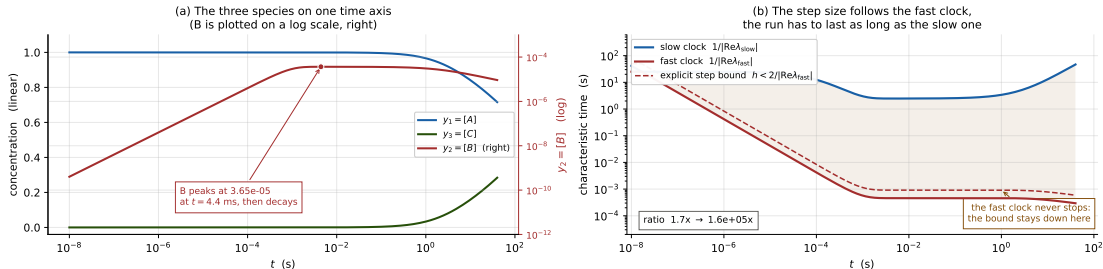
Now A and C

A drifts down from 1 to 0.7158; C drifts up from 0 to 0.2842. Both of these move on a scale of **tens of seconds** — a completely different world from B.

F2: the reference solution, time on a logarithmic axis.

Why it is hard: B is a clock that ticks in microseconds

F13 What the system looks like, and the two clocks that make it expensive

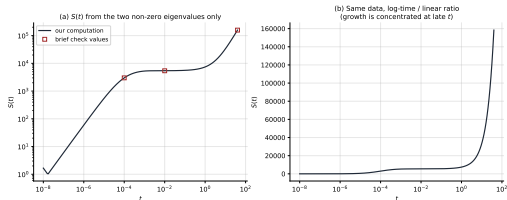


F13: concentrations (left) and their clocks (right), i.e. characteristic times $1/|\text{Re} \lambda|$.

B is extremely reactive: the moment it forms it is consumed within microseconds, so its concentration never leaves the 10^{-5} range. The system therefore carries **two clocks of wildly different rate** — only $1.7 \times$ apart at the start, but $1.58 \times 10^5 \times$ apart by $t = 40$. On the next slide we put a number on that ratio and give it a name.

Why it is hard (1): how different are the two clocks? — stiffness

F6 Stiffness ratio $S(t) = \max|\operatorname{Re}\lambda|/\min|\operatorname{Re}\lambda|$ over the two non-zero eigenvalues



F6: $S(t) = \max |\operatorname{Re} \lambda| / \min |\operatorname{Re} \lambda|$, counting only the two non-zero eigenvalues

A basic concept first: what is stiffness?

At any time t , the eigenvalues λ of $J(t)$ give the time scales $\tau = 1/|\operatorname{Re} \lambda|$ of each mode, so

$$S(t) = \frac{\max |\operatorname{Re} \lambda|}{\min |\operatorname{Re} \lambda|}$$

is the ratio of the fast and slow clocks. When $S \gg 1$ the problem is **stiff**.

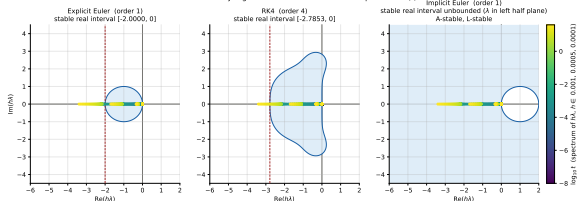
t	our S	reference value
10^{-4}	3.0097×10^3	3.0×10^3
10^{-2}	5.4221×10^3	5.4×10^3
40	1.5840×10^5	1.58×10^5

The stiffest moment is $t = 40$, not $t = 0$

The three points agree to 0.25%–0.41%; at $t = 10^{-8}$ the ratio is only 1.67, while at $t = 40$ it is 1.58×10^5 . So the step-size limit must use the λ at $t = 40$, not at the start.

Why it is hard (2): explicit methods have a short stability interval

F4 Absolute stability regions and the actual Robertson spectrum $h\lambda(t)$



F4: absolute stability regions of the three methods

Another basic concept: absolute stability region

Write one step as $y_{n+1} = R(z)y_n$ with $z = h\lambda$. The method is **absolutely stable** when $|R(z)| \leq 1$ (errors are not amplified). The set of such z is the stability region.

- For explicit methods R is a *polynomial*, so the region is **bounded**.
- For implicit Euler $R(z) = 1/(1 - z)$ and $|R| \leq 1$ everywhere in the left half-plane.

method	real-axis interval	order
Explicit Euler	$[-2.0000, 0]$	1
RK4	$[-2.7853, 0]$	4
Implicit Euler	$(-\infty, 0]$	1

Substituting $z = h\lambda$ gives $h_{\text{EE}} < 5.895 \times 10^{-4}$, $h_{\text{RK4}} < 8.209 \times 10^{-4}$; implicit Euler has **no limit** in this direction.

Consequence: the upper bound nails the cost

A hard lower bound on cost

Use $|\lambda|_{\max} = 3392.79$ (at $t = 40$) in $h < 2/|\lambda|$:

$$h < 5.9 \times 10^{-4} \text{ s}$$

Crossing $[0, 40]$ therefore needs at least

$$N \geq \frac{40}{5.9 \times 10^{-4}} \approx 6.8 \times 10^4$$

This is a *lower bound*, not a recommendation: explicit methods cannot even fill this many steps — the measured largest stable step is 8% smaller (Conclusion 2).

The awkward part: the limit is at the end but governs all

$|\lambda|_{\max}$ is the maximum over the *whole interval*, and it occurs at $t = 40$, not at the start. Long after B has settled and nothing in the system is changing quickly, the restriction is **still there**. The fast process *pins the global step size*.

Why we cannot “only be careful at the beginning”

Stability is a *cumulative* test: once a step crosses the boundary, $|R(z)| > 1$ and the error is amplified at every subsequent step. Raising h from 5.4159×10^{-4} by only 15% already produces a concentration of **-182** at $t = 32.9$.

How we did it: build a ruler, then measure three methods

Step one: build a ruler

Compute a **reference solution** with a high-accuracy solver, then confirm it by three independent routes, agreeing to **10 digits**. *Without that ruler the words “how accurate” mean nothing.*

Step two: three methods, and check their orders first

Explicit Euler (1) **RK4** (4) **Implicit Euler** (1). All written from scratch; then we measure their *observed orders* to verify that the formulas are correct.

Step three: put every method through four checks

Conservation / non-negativity / accuracy / endpoint. The four must be read **separately** — they do *not* imply one another, and the biggest trap in this problem is hidden in one of the combinations. (How each is measured is on the next slide.)

Division of labour

Ruler (independent) → methods (ours) → checks (separate) → conclusions (each directly verifiable from the data above).

Step one: build a ruler we can trust

Three mutually independent confirmations (T4)

- **Tolerance repetition:** rerun the same solver at tighter tolerances and see at which digit the numbers stop moving
- **Cross-validation:** compare against SciPy's BDF and LSODA
- **Structural identity:** $m2(J)$ must equal $\det(B_{\text{red}})$

The three routes fail in different ways (step size, implementation, algebra), so agreement among them is meaningful.

Confirmation route	agreeing digits
tolerance repetition	10.76
vs BDF	10.70
vs LSODA	10.98
lower bound used	10.76

Largest componentwise deviation 2.089×10^{-12} .

This is the yardstick for everything that follows

The reference gives $y_2(40) = 9.185535 \times 10^{-6}$, quotable to 10 digits. Every error quoted later is an **pointwise comparison against it**.

Step two: what the three methods actually are

Explicit Euler (1st order)

$$y_{n+1} = y_n + h f(t_n, y_n)$$

The right-hand side uses only the known y_n , so one step costs one evaluation of f . The amplification factor is $R(z) = 1 + z$.

RK4 (4th order)

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

A weighted average of four slopes, four evaluations of f per step: **four times the work for fourth-order accuracy**.

Implicit Euler (1st order)

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$$

The unknown appears on *both* sides: each step solves

$$G(u) = u - y_n - h f(u) = 0$$

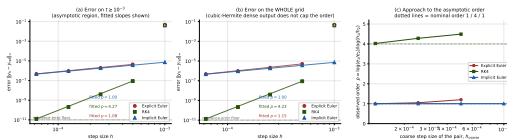
with Newton's method, requiring J and one linear solve per step.

The difference is only two things

(1) how many f evaluations per step (work); and (2) *whether the unknown future value is used* (stability). RK4 is good at (1) and bad at (2); implicit Euler is the opposite. All later conclusions are intersections of these two lines.

Step two (cont.): first check that the observed orders are right

F8: Uniform-step convergence, observed order and its limits



F8: error vs h on a log-log scale

How we measure it: repeatedly halve h , measure the error, and read the slope on the log-log plot.

Basics: on a log-log plot the slope is the order

If the error $\approx Ch^p$, then $\log(\text{err})$ vs $\log h$ is a straight line of slope p . So the order is something we can **measure**, not just trust from a formula.

method	nominal	fitted slope	R^2
EE	1	1.079 ± 0.033	0.9982
RK4	4	4.268 ± 0.074	0.9994
IE	1	$0.99996 \pm 1.2 \times 10^{-5}$	1.0000

All three are first interpolated by cubic Hermite onto the same 201-point grid before comparison. If the interpolation were only $O(h^4)$, the observed RK4 order would be capped at 2; the measured 4.27 shows it **is not**.

Step three: what the four checks measure, and why none can be skipped

1 Conservation

measured: $\max_t |y_1 + y_2 + y_3 - 1|$, every run

why separately: in this problem the sum is **structurally exact**, so this check has **no discriminating power** (see Conclusion 1)

3 Accuracy

measured: pointwise maximum error against the reference

why separately: the only check that really measures “accuracy” — and the only one that can detect “stable but wrong”

2 Non-negativity

measured: $\min_i y_i$, every run

why separately: concentrations are physical quantities; $y_i < 0$ means the solution already has *no meaning*, even if conservation looks fine

4 Endpoint

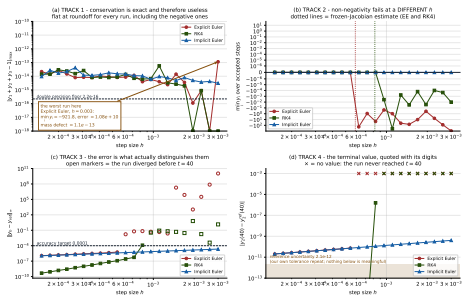
measured: $y_2(40)$ against the reference value

why separately: it is the requested number, but it reflects one point only and **cannot represent the whole interval**

1 **2** **3** pass while **4** is far off: the four checks are **logically independent** — each can be true or false on its own ($2^4 = 16$ combinations). The next slide nails this biggest trap with data.

Conclusion 1: among the four tracks, only one catches the error

F9 Four diagnostics, four verdicts: conservation cannot stand in for accuracy



F9: conservation / non-negativity / error / endpoint, four tracks

What we did: for each method, run 22 logarithmically spaced step sizes, recording the four quantities simultaneously. Which track fails *first* is immediately visible.

The most glaring run (explicit Euler, $h = 3 \times 10^{-3}$)

conservation defect = 1.1×10^{-13} (looks perfect)

$\min_i y_i = -921.8$ (concentration is negative!)

error = 1.08×10^{10}

Check conservation only and this garbage passes.

The four tracks are not synchronized

- the conservation defect is $\sim 10^{-14}$ at every step size — **no discriminating power at all**
- RK4 has a whole window that is stable, non-negative and *already inaccurate*
- the track that fails first is *not the same* for each method: EE loses accuracy and non-negativity together, RK4 loses accuracy first

Conclusion 1, why? Why is the conservation defect always 10^{-14} ?

Step one: add the three equations

Every term on the right cancels in pairs: $-0.04y_1$ with $+0.04y_1$, $+10^4y_2y_3$ with $-10^4y_2y_3$, $-3 \times 10^7y_2^2$ with $+3 \times 10^7y_2^2$. Hence

$$\frac{d}{dt}(y_1 + y_2 + y_3) \equiv 0 \implies S \equiv 1$$

S is an *exact invariant* of the system, independent of the numerical method.

Step two: all three schemes preserve it automatically

Explicit Euler: adding the components gives $S_{n+1} = S_n + h \sum_i f_i = S_n$ because $\sum_i f_i \equiv 0$. The same happens for every RK4 stage and for implicit Euler. **This holds for any step size and any order.**

So it only measures floating-point round-off

10^{-14} is not “good accuracy”; it is the magnitude of round-off in double precision. A quantity that is **exactly preserved by construction** cannot diagnose accuracy — it has no discriminating power, not because the numerics are good, but because *it cannot structurally fail*.

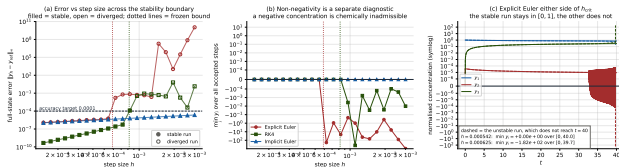
A general rule of thumb

Whenever you see an “invariant check”, ask first: *is it preserved by the discretisation, or is it affected by discretisation error?* The former only catches implementation bugs; it will not catch accuracy problems.

In this problem the checks that can actually fail are non-negativity ($\min_i y_i$), accuracy (pointwise error) and the endpoint ($y_2(40)$).

Conclusion 2: the step-size limit can be predicted in advance, and accurately

F7 Frozen-Jacobian estimate vs an empirical step sweep



F7: 22-point logarithmic sweep vs the frozen-jacobian bound

What we did: first *compute* a predicted bound from $J(t)$ (the derivation is on the next slide), then actually run a 22-point sweep and see where the solution breaks. These are **two independent things**: prediction from linear algebra, measurement from real runs.

units of 10^{-4}	EE	RK4
predicted (frozen bound)	5.895	8.209
largest stable h	5.4159	8.3088
first divergent h	6.2464	9.5827

The measured boundary falls **between two adjacent sweep points**: EE fails 8% earlier than predicted, RK4 holds 1.2% longer. A one-page derivation therefore predicts the usable step size to within about ten percent.

Stated honestly

This is a *locally linear*, frozen estimate, while $J(t)$ moves and the system is strongly non-normal. So it is a diagnostic, not a theorem.

Conclusion 2, why? Where does the bound come from

Idea one: “freeze” the nonlinear problem at one instant

At some time t , treat $J(t)$ as *constant*. Then $y' = Jy$ has solution e^{Jt} , and one step has amplification factor $R(h\lambda)$. Requiring **all** λ to satisfy $|R(h\lambda)| \leq 1$, the most restrictive one gives the step-size limit.

Substitute $|\lambda|_{\max} = 3392.79$ (at $t = 40$)

$$h_{\text{EE}} \leq \frac{2.0000}{3392.79} = 5.895 \times 10^{-4}$$

$$h_{\text{RK4}} \leq \frac{2.7853}{3392.79} = 8.209 \times 10^{-4}$$

Idea two: the two amplification polynomials

method	$R(z)$	real-axis limit
EE	$1 + z$	2.0000
RK4	$1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$	2.7853

R is a polynomial for explicit methods $\Rightarrow |R| \leq 1$ only on a short real interval $\Rightarrow h \leq \text{limit}/|\lambda|_{\max}$.

Why this single bound governs the whole interval

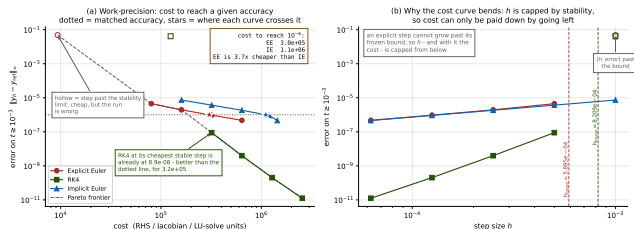
We use the *global maximum* $|\lambda|_{\max}$. The bound at any other time is never stricter, so one number covers $[0, 40]$. Using the $t = 0$ eigenvalue would overestimate the limit by more than 10^5 .

Why the measurement is not exactly equal to it

J varies and the system is strongly non-normal, so an 8% deviation is normal. **Being within ten percent is already useful.**

Conclusion 3: implicit stability does not buy cheaper computation

F12 Cost at matched accuracy - the implicit method is not automatically cheaper



F12: who is more expensive *at matched accuracy*

What we did: run each method at five step sizes, plot the “cost–error” curves, then cut horizontally at several *target errors* and compare the minimum cost each method needs to reach that accuracy. **We compare equal accuracy, not equal step size.**

target error	EE	RK4	IE
10^{-5}	0.80	3.2	1.6
3×10^{-6}	1.1	3.2	3.9
10^{-6}	3.0	3.2	11
10^{-7}	—	3.2	—

Units of 10^5 cost units (defined on the next slide).

Three readings

- At every target IE can reach, it costs **2.0–3.7×** more than EE
- At 10^{-7} , neither EE nor IE reaches it within the swept range; RK4 is already there at its cheapest stable step
- the cost floor is pinned by $40/h$, and h is capped by the stability bound

Conclusion 3, why? Why “more stable” is not “cheaper”

How cost is counted

We use a single *cost unit*: one right-hand-side evaluation f counts as 1, one Jacobian evaluation counts as 1, and one linear solve (LU) counts as 1; add them up.

- Explicit Euler: one step = 1 evaluation of $f \Rightarrow 1$
- RK4: one step = 4 evaluations of $f \Rightarrow 4$
- Implicit Euler: one step = $f + J + \text{solve} \Rightarrow 3$

Why this accounting is necessary

The three methods do different amounts of work per step, so comparing “number of steps” would bias the result toward explicit methods. Expressing every operation in the same unit is the only way to ask “who is cheaper at the same accuracy”.

Implicit Euler loses on both sides of the product

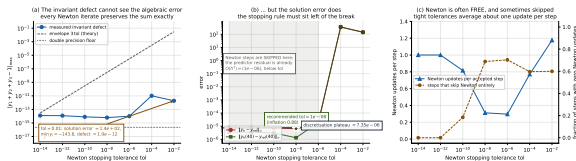
- Implicit Euler is 1st order: to hit an error of 10^{-6} it needs a small h , so the number of steps $40/h$ stays up
- Each step still costs 3 units, three times explicit Euler
- Multiply the two and at 10^{-6} IE costs **3.66×** as much as EE

Why RK4 wins at the end

RK4 costs 4 units per step (more than IE), but it is 4th order and its step-size limit is also larger (8.209×10^{-4} vs the small h forced on IE by accuracy). At 10^{-7} RK4 reaches the target at its cheapest stable step, while the others would need even smaller h and still higher cost.

Conclusion 4 (top): the Newton tolerance has an upper bound

F10 Newton stopping-tolerance sweep at $h = 0.001$: the algebraic error must be negligible against the discretisation error



F10: conservation defect (flat) vs solution error (not flat)

Basics: implicit Euler must solve $G(u) = u - y_n - hf(u) = 0$ each step. The standard predictor is $u^0 = y_n + hf(y_n)$, which is exactly one explicit-Euler step. If the residual $\|G(u^0)\| \leq \text{tol}$, the predictor is accepted and **Newton takes zero updates**.

Why a small h makes the residual small automatically

Substituting the predictor gives $G(u^0) = -\frac{h^2}{2} J_f f + O(h^3)$, i.e. the residual is $O(h^2)$ (about 10^{-6} when $h = 10^{-3}$). So there is a threshold

$$h^* = \sqrt{\text{tol} / \|J_f f\|_\infty}$$

Using $\|J_f f\|_\infty = 8.2324 \times 10^{-3}$:

tolerance	error inflation	h^*
10^{-6}	2.67	1.1×10^{-2}
10^{-8}	0.88	1.1×10^{-3}
10^{-10}	1.00	1.1×10^{-4}

10^{-8} suffices; tighter is waste. But 10^{-6} pushes h^* past the explicit stability bound 5.9×10^{-4} by **nineteen times**, and *the error estimator sees nothing*.

Conclusion 4, why? Loosening the tolerance changes the method

tolerance	error inflation	zero-update fraction
10^{-2}	1.96×10^7	60.0%
10^{-6}	2.67	70.3%
10^{-8}	0.88	68.8%
10^{-10}	1.00	18.4%

The third column is the fraction of steps whose predictor was accepted without any Newton update. 10^{-8} and 10^{-6} look similar (68.8% vs 70.3%), but the error inflation differs by a factor of 3 — **this fraction alone does not diagnose the problem.**

An independent second test (T18)

At a fixed state, trial by trial: $\text{tol} = 10^{-14}$ gives **0/7** purely explicit steps; $\text{tol} = 10^{-6}$ gives **7/7** purely explicit steps — the method is genuinely changed.

Why the error estimator does not notice

Adaptive / dense-output estimators use $E = \|y_{\text{whole}} - y_{\text{two halves}}\|$. This is a *difference between two sides*; as long as both sides use the same scheme it gives the expected $O(h^2)$ signal and the controller is happy.

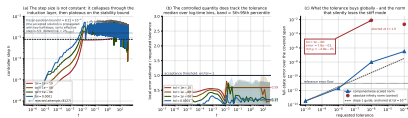
- Fully explicit and fully Newton-corrected runs at the same h give E values within 1% of each other
- The fitted h -exponent stays inside $[1.89, 2.10]$ — no anomaly

The take-away: a silent method switch

The tolerance is not “the tighter the better”; it must be tight enough that Newton is **actually working**. Here 10^{-8} is right. The right test is the computable h^* , not the error estimator after the fact.

Conclusion 4 (bottom): adaptivity is stability-limited, not accuracy-limited

F11 RK4 step-doubling controller: step size, error tracking, and a norm that fails



F11: $h(t)$, err/tol trace, and a failing norm

Basics: adaptivity compares one whole step with two half steps, then scales h to meet tol. Accepted steps use **two half steps**, so the stability limit applies to $h/2$.

tol	$h/2 \div$	bound	err/tol
10^{-4}		1.07	0.17
10^{-6}		1.07	0.19
10^{-8}		1.06	0.31
10^{-10}		0.83	0.59

At looser tolerances

$h/2$ sits 6–7% above the frozen bound while err/tol is only 0.17–0.31: **the accuracy budget is nowhere near spent.**

Another trap: the absolute L^∞ controller is wrong

y_2 is $O(10^{-5})$ but the error scale is 10^{-10} . With an absolute tolerance 10^{-6} the controller accepts steps far beyond the RK4 stability limit:

- tol = 10^{-6} : runs to the end, but y_2 reaches -4.77×10^{-5} , total error 7.83×10^{-3}
- tol = 10^{-4} : **aborts** at $t = 1.94$ with $\min y = -9.03 \times 10^2$

The fix

Use a **scaled (mixed) norm**: $\max_i |e_i| / (\text{atol}_i + |y_i|) \leq \text{tol}$. **T12** reports both norms.

Four conclusions, one number each

conclusion	the number	how to read it
1 four tracks disagree	conservation defect 1.1×10^{-13} , but $\min_i y_i = -921.8$	conservation is structurally exact here, so it has no power; checking it alone would let through a run with error 1.08×10^{10}
2 bound can be predicted	predicted 5.895×10^{-4} , measured stable limit 5.4159×10^{-4}	8% apart — good enough as a diagnostic, not a theorem
3 implicit is not cheaper	at 10^{-6} error: EE 3.0, IE 11 (10^5 cost units)	stability is bought, but at $3.66\times$ cost; at 10^{-7} RK4 is cheapest
4 both knobs have upper bounds	Newton tol 10^{-8} suffices; 10^{-6} overshoots the bound $19\times$	adaptive $h/2$ is pinned by the stability bound, err/tol only 0.17

The number we hand in

$y_2(40) = 9.185535 \times 10^{-6}$, quotable to **10 digits**; the peak 3.6487×10^{-5} at $t = 4.42 \times 10^{-3}$ is 4 times larger than the endpoint, so we report both.

One-sentence summary

Root of the difficulty: two clocks, and the fast one pins everything

The two clocks start only $1.7\times$ apart, but by $t = 40$ they differ by $1.58 \times 10^5\times$. The fast one pins the explicit step size at $h < 5.9 \times 10^{-4}$, so crossing $[0, 40]$ needs at least 6.8×10^4 steps — *and this restriction does not relax even after the fast process has died down*. That is stiffness.

Methodological lesson: validation is easier to fool than solving

The conservation defect is 1.1×10^{-13} (looks perfect), yet the same run has concentration -921.8 and error 1.08×10^{10} . “Did not blow up” and “passed a check” are **not the same as “right”**. The reason is that conservation is *exactly* preserved by the discretisation, so it has no discriminating power; only an independently built ruler (three confirmation routes agreeing to 10.76 digits) compared point by point counts.

“More stable” and “cheaper” are two different things

“Implicit is more stable” is true; “implicit is cheaper” is **not true** in the reachable accuracy range — at 10^{-6} error EE costs 3.0×10^5 cost units while IE costs 1.1×10^6 . You only see this by putting *cost, error and stability bound* on the same plot.

Reproducibility and AI transparency

One command recomputes everything

```
python run_all.py (about 6 minutes, 10 stages)
reference / eigen / stability / integrators
/ sweep /
diagnostics / newtonsweep / adaptive /
newtonstudies / figures
```

- results/ contains **no hand-written numbers**
- SciPy is used *only* as a reference-check; EE / RK4 / IE / Newton are all **implemented from scratch**
- 148 tests pass, including several *deliberate negative-result* regression tests

AI transparency log (18 entries; full list in appendix/ai_transparency_log.md)

(1) reduced Jacobian taken as a submatrix · (2) significant-digit criterion made unusable · (3) RK4 real-axis stability polynomial written wrong · (4) assumption that a fixed grid resolves eigenvalue crossings · (5) uniform first step swallowed the whole interval · (6) inexact-Newton claim copied without quantification · (7) measured stability boundary assumed to hit frozen bound · (8) Newton tolerance treated as an infinitely tight knob · (9) absolute-norm controller misses the stiff mode · (10) conservation treated as a universal accuracy gate · (11) a slide number disagreed with its table · (12) first slide deck followed computation order and was unreadable · (13) the same wrong number stayed in the README · (14) three conservation-defect values in the adaptive table did not match **T12** · (15) damped-Newton test written on the wrong system, Armijo constant off by $2\times$ · (16) Jacobian checker transposed the finite-difference matrix and falsely rejected correct matrices · (17) “loose tolerance only loses accuracy” is wrong — it changes the method, and the estimator cannot see it · (18) **T16** measured a cold start but was described as a driven path — table right, caption wrong

Every entry records what was wrong, how it was found, and what was changed.